

Session 20 (AIML) -Optional Task

- **Name of Student** : Shruti Dnyaneshwar Darwatkar
- **Domain Name:** AIML Development
- **College Name:** Zeal Polytechnic , Narhe
- **Branch Name:** AIML
- **Github url:** https://github.com/shrutidarwatkar-sketch/Task_ITR.git

Step 1: Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import joblib

from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score
```

Step 2: Load Dataset

```
from google.colab import files

uploaded = files.upload()
```

... Choose files laptop_data.csv
laptop_data.csv(text/csv) - 182053 bytes, last modified: 15/07/2026 - 100% done
Saving laptop_data.csv to laptop_data (1).csv

Display first rows

df.head()

	Company	TypeName	Inches	ScreenResolution	Cpu	Ram	Memory	Gpu	OpSys	Weight	Price
0	1	4	13.3	23	65	8	4	58	8	38	71378.6832
1	1	4	13.3	1	63	8	2	51	8	35	47895.5232
2	7	3	15.6	8	74	8	16	53	4	74	30636.0000
3	1	4	15.4	25	85	1	29	9	8	71	135195.3360
4	1	4	13.3	23	67	8	16	59	8	38	96095.8080

Dataset information

df.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1303 entries, 0 to 1302
Data columns (total 11 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   Company                1303 non-null   int64
1   TypeName               1303 non-null   int64
2   Inches                 1303 non-null   float64
3   ScreenResolution       1303 non-null   int64
4   Cpu                    1303 non-null   int64
5   Ram                    1303 non-null   int64
6   Memory                 1303 non-null   int64
7   Gpu                    1303 non-null   int64
8   OpSys                  1303 non-null   int64
9   Weight                 1303 non-null   int64
10  Price                  1303 non-null   float64
dtypes: float64(2), int64(9)
memory usage: 112.1 KB
```

Dataset shape

df.shape

(1303, 11)

Describe dataset

```
df.describe()
```

	Company	TypeName	Inches	ScreenResolution	Cpu	Ram	Memory	Gpu	OpSys	Weight	Price
count	1303.000000	1303.000000	1303.000000	1303.000000	1303.000000	1303.000000	1303.000000	1303.000000	1303.000000	1303.000000	1303.000000
mean	6.442824	2.582502	15.017191	9.224098	76.983883	5.725249	16.345357	54.682272	4.808903	90.465081	59870.042910
std	4.084128	1.245231	1.426304	7.885050	25.190467	2.630268	8.008281	21.623195	1.035253	43.026196	37243.201786
min	0.000000	0.000000	10.100000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	9270.720000
25%	4.000000	1.500000	14.000000	6.000000	65.000000	5.000000	10.000000	46.000000	5.000000	53.500000	31914.720000
50%	7.000000	3.000000	15.600000	8.000000	78.000000	7.000000	16.000000	53.000000	5.000000	96.000000	52054.560000
75%	10.000000	3.000000	15.600000	15.000000	97.000000	8.000000	23.000000	72.000000	5.000000	122.000000	79274.246400
max	18.000000	5.000000	18.400000	39.000000	117.000000	8.000000	38.000000	109.000000	8.000000	178.000000	324954.720000

Step 3: Check Missing Values

```
df.isnull().sum()
```

...	0
Company	0
TypeName	0
Inches	0
ScreenResolution	0
Cpu	0
Ram	0
Memory	0
Gpu	0
OpSys	0
Weight	0
Price	0

dtype: int64

If any missing values exist

```
df.dropna(inplace=True)
```

Step 4: Check Duplicate Records

```
df.duplicated().sum()  
np.int64(29)
```

Remove duplicates

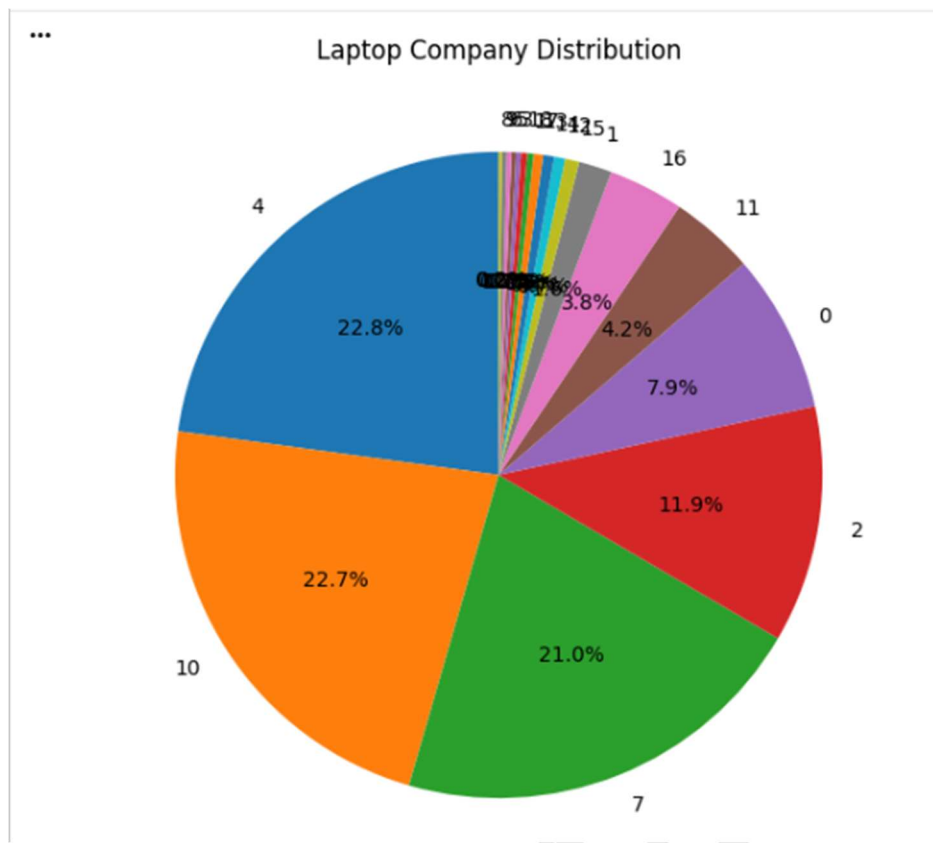
```
df.drop_duplicates(inplace=True)
```

Step 5: Display Column Names

```
df.columns  
  
... Index(['Company', 'TypeName', 'Inches', 'ScreenResolution', 'Cpu', 'Ram',  
          'Memory', 'Gpu', 'OpSys', 'Weight', 'Price'],  
          dtype='object')
```

Step 6: Pie Chart (Company Distribution)

```
company = df['Company'].value_counts()  
  
plt.figure(figsize=(7,7))  
plt.pie(company,  
        labels=company.index,  
        autopct="%1.1f%%",  
        startangle=90)  
  
plt.title("Laptop Company Distribution")  
plt.show()
```

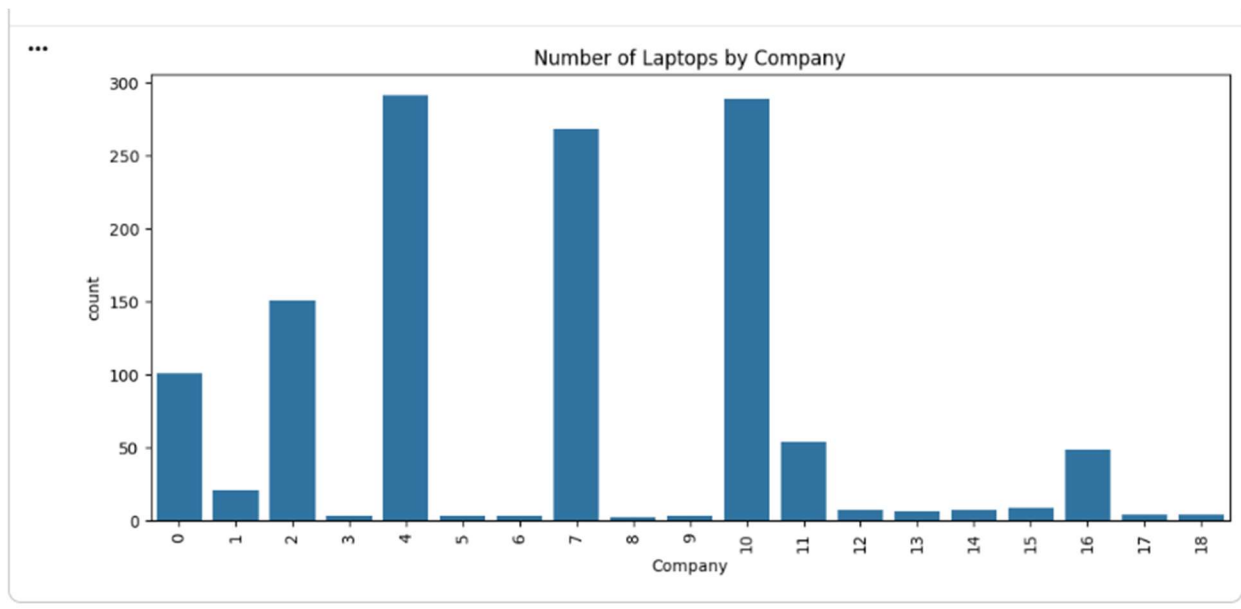


Step 7: Bar Chart (Number of Laptops by Company)

```
plt.figure(figsize=(12,5))

sns.countplot(data=df,
              x='Company')

plt.xticks(rotation=90)
plt.title("Number of Laptops by Company")
plt.show()
```



Step 8: Histogram (Price Distribution)

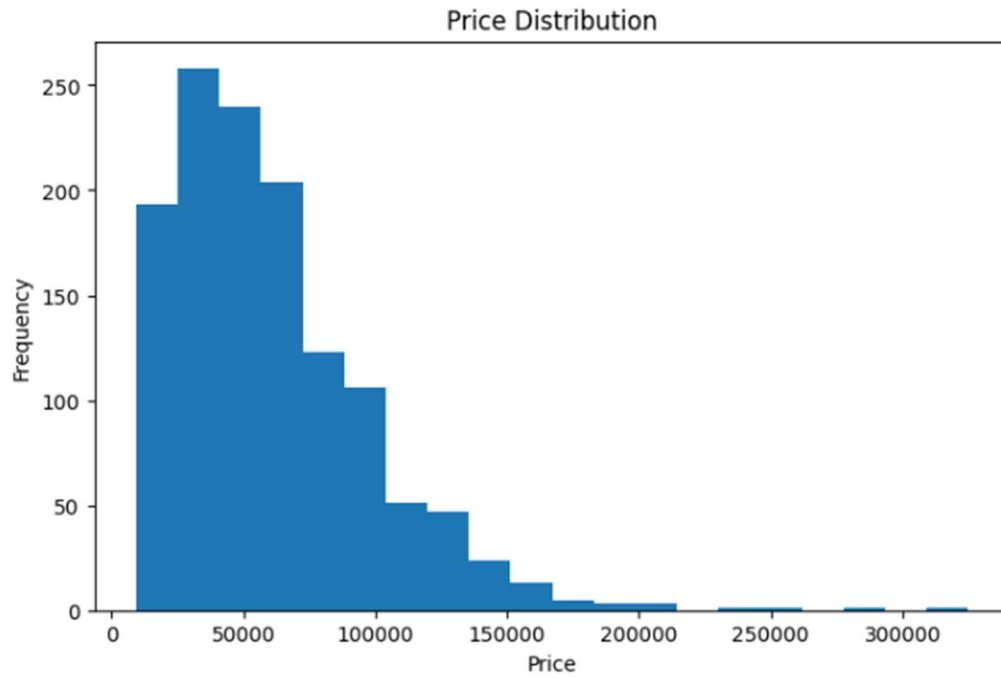
```
plt.figure(figsize=(8,5))

plt.hist(df['Price'], bins=20)

plt.title("Price Distribution")
plt.xlabel("Price")
plt.ylabel("Frequency")

plt.show()
```

...



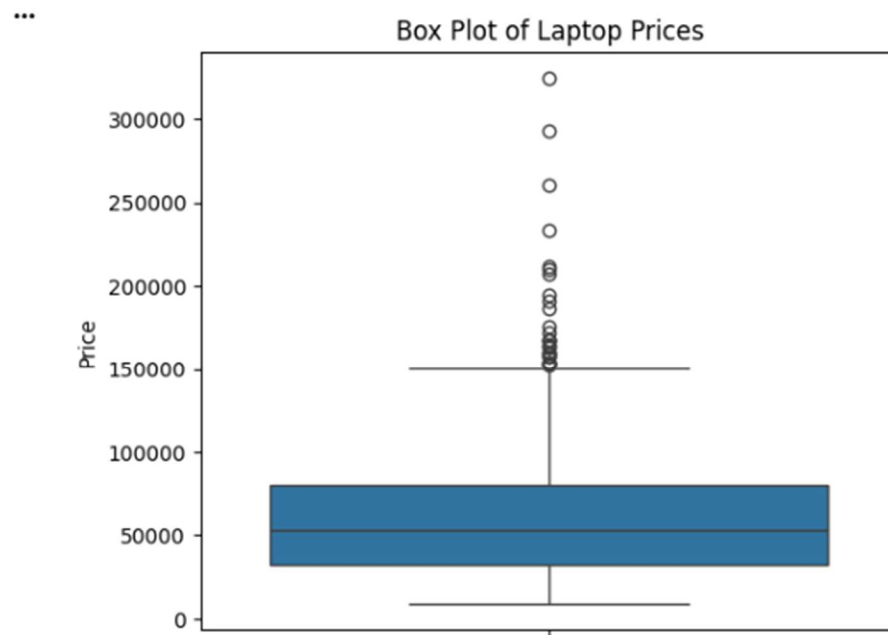
Step 9: Box Plot

```
plt.figure(figsize=(6,5))

sns.boxplot(y=df['Price'])

plt.title("Box Plot of Laptop Prices")

plt.show()
```



Step 10: Scatter Plot (RAM vs Price)

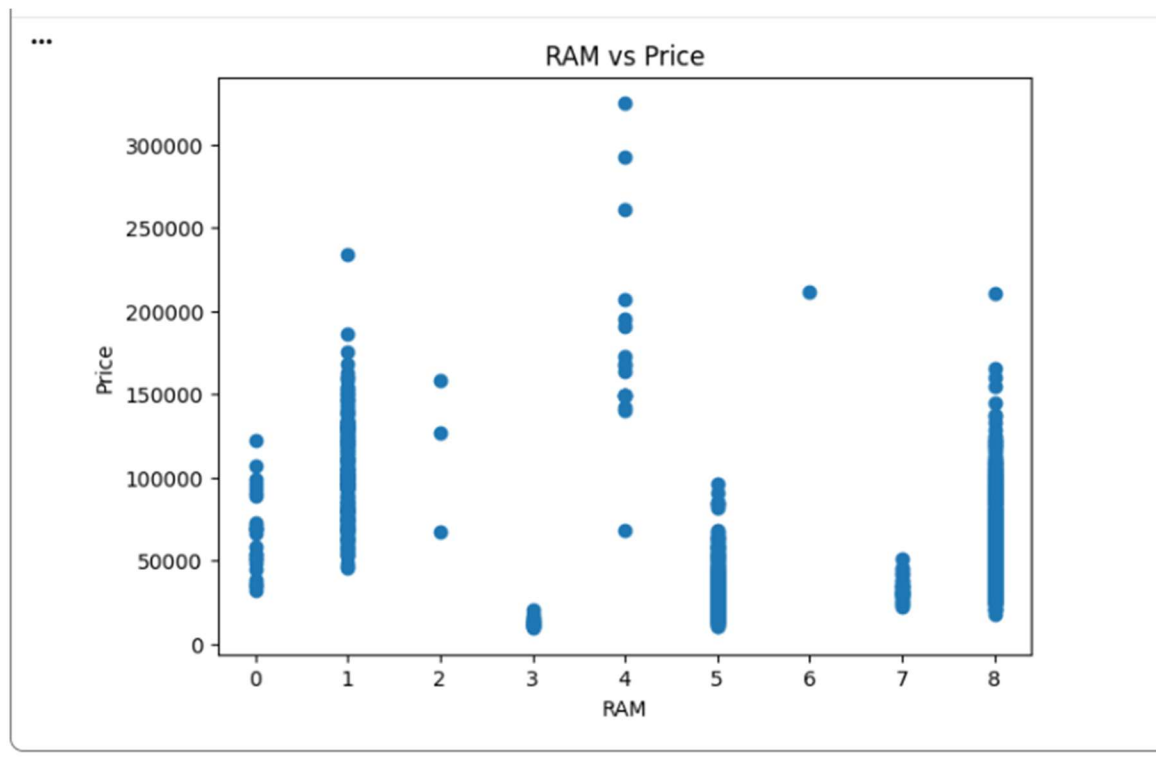
```
plt.figure(figsize=(7,5))

plt.scatter(df['Ram'],
            df['Price'])

plt.xlabel("RAM")
plt.ylabel("Price")

plt.title("RAM vs Price")

plt.show()
```



Step 11: Count Plot (Operating System)

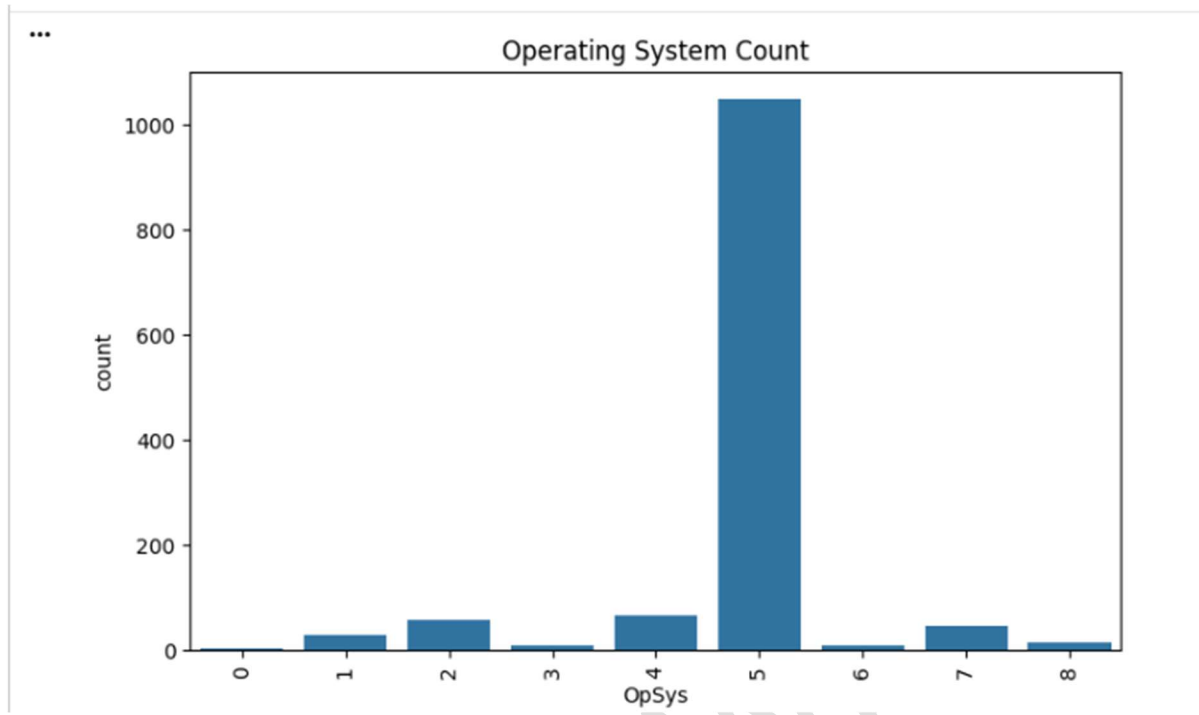
```
plt.figure(figsize=(8,5))

sns.countplot(data=df,
              x='OpSys')

plt.xticks(rotation=90)

plt.title("Operating System Count")

plt.show()
```



Step 12: Count Plot (Laptop Type)

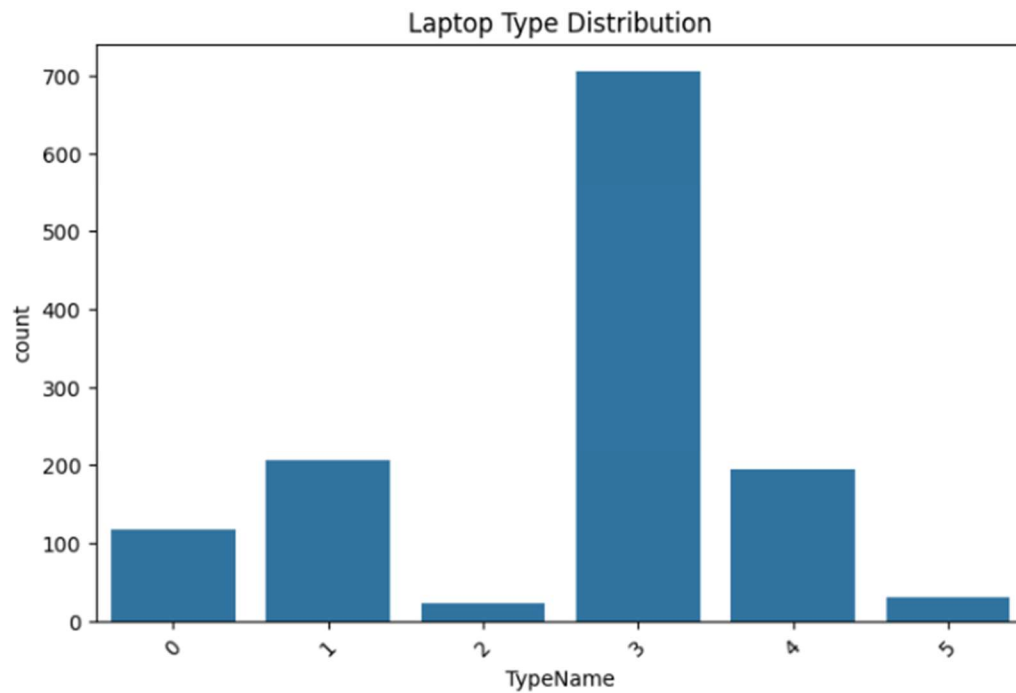
```
plt.figure(figsize=(8,5))

sns.countplot(data=df,
              x='TypeName')

plt.xticks(rotation=45)

plt.title("Laptop Type Distribution")

plt.show()
```



Step 13: Correlation Heatmap

```
le = LabelEncoder()

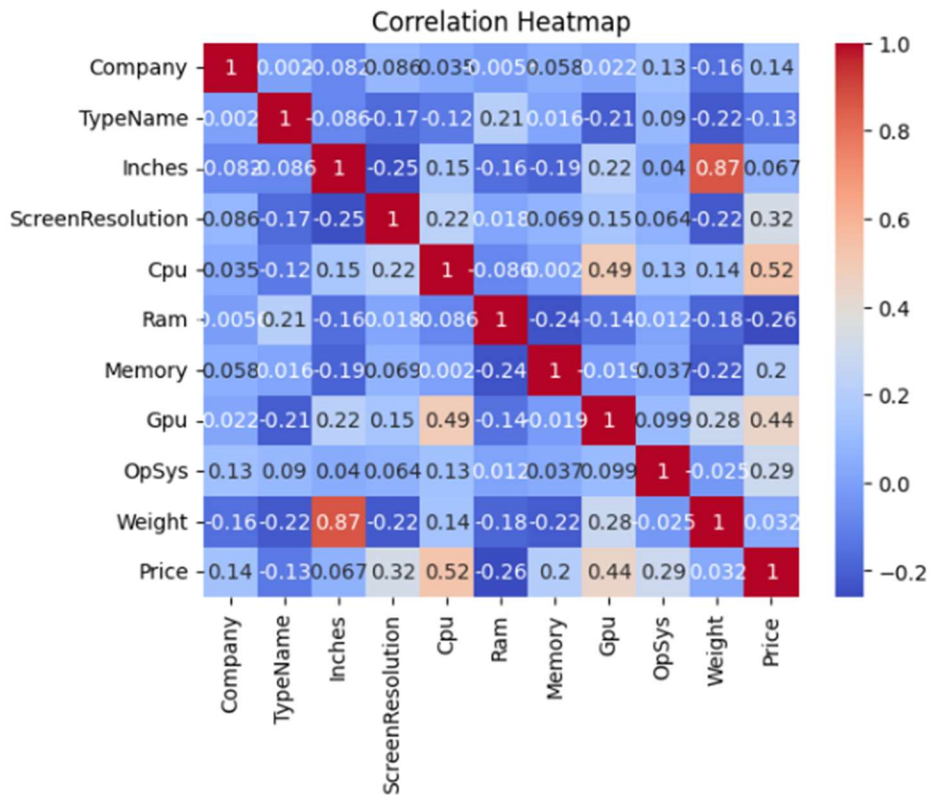
for col in df.select_dtypes(include='object').columns:
    df[col] = le.fit_transform(df[col])
plt.figure(figsize=(12,10))

sns.heatmap(df.corr(),
            annot=True,
            cmap='coolwarm')

plt.title("Correlation Heatmap")

plt.show()
```

...



Step 14: Features and Target

```
# Independent Features
X = df.drop("Price", axis=1)

# Dependent Feature
y = df["Price"]

print("Shape of X:", X.shape)
print("Shape of y:", y.shape)
```

... Shape of X: (1274, 10)
Shape of y: (1274,)

Step 15: Label Encoding

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()

for col in X.select_dtypes(include='object').columns:
    X[col] = le.fit_transform(X[col])

print(X.head())
```

```
...   Company  TypeName  Inches  ScreenResolution  Cpu  Ram  Memory  Gpu  OpSys  \
0         1         4    13.3             23    65    8         4    58    8
1         1         4    13.3             1    63    8         2    51    8
2         7         3    15.6             8    74    8        16    53    4
3         1         4    15.4            25    85    1        29    9    8
4         1         4    13.3            23    67    8        16    59    8

Weight
0     38
1     35
2     74
3     71
4     38
```

Step 16: Feature Scaling

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X = scaler.fit_transform(X)
```

Step 17: Train-Test Split

```

▶ from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.33,
    random_state=42
)

print(X_train.shape)
print(X_test.shape)
print(y_train.shape)
print(y_test.shape)

```

```

... (853, 10)
    (421, 10)
    (853,)
    (421,)

```

Step 18: Build Model

```

▶ from sklearn.linear_model import LinearRegression

model = LinearRegression()

model.fit(X_train, y_train)

print("Intercept:", model.intercept_)
print("Coefficients:", model.coef_)

```

```

... Intercept: 60340.94603667729
Coefficients: [ 1828.68436909  254.43936187  6372.67075159  7039.57961159
 12096.01850565 -6981.62742427  4976.32921204  7568.513988
 7101.96530376 -7383.41238185]

```

Step 19: Prediction

```
▶ y_pred = model.predict(X_test)

comparison = pd.DataFrame({
    "Actual Price": y_test.values,
    "Predicted Price": y_pred
})

comparison.head(10)
```

```
***
```

	Actual Price	Predicted Price
0	24988.3200	51010.379175
1	41345.2800	42116.001567
2	63882.7200	80512.655077
3	42357.6000	51531.760784
4	68184.0144	53695.754444
5	38308.3200	44819.878296
6	111355.2000	81270.210067
7	31808.1600	22387.287105
8	51729.5520	47100.030807
9	36336.9600	69217.424839

Step 20: R² Score

```
▶ from sklearn.metrics import r2_score

score = r2_score(y_test, y_pred)

print("R2 Score:", score)

*** R2 Score: 0.4213646175244068
```

Step 8: Save Model

```
joblib.dump(model, "Laptop_Model.pkl")
joblib.dump(scaler, "Scaler.pkl")
joblib.dump(X_train.shape, "Columns.pkl")

print("Model Saved Successfully")

... Model Saved Successfully
```

❖ Conclusion :

- Successfully imported all the required Python libraries.
- Loaded the laptop dataset into a Pandas DataFrame.
- Explored the dataset using head(), info(), shape, and describe().
- Checked and handled missing values and duplicate records.
- Visualized the data using Pie Chart, Bar Chart, Histogram, Box Plot, Scatter Plot, Count Plot, and Correlation Heatmap.
- Converted categorical features into numerical values using Label Encoding.
- Separated the independent features (X) and target feature (Price).
- Standardized the features using StandardScaler.
- Split the dataset into training and testing sets using train_test_split().
- Built and trained a Linear Regression model.
- Predicted laptop prices using the trained model.
- Compared the predicted prices with the actual prices.
- Evaluated the model using the R² Score, which measures prediction accuracy.
- Saved the trained model, scaler, and feature information using Joblib.
- Successfully created a Laptop Price Prediction Machine Learning model that can be used to predict laptop prices for new data.